

Módulo 8 – Capítulo 01 – Sección 01

Open weights vs open source: qué significa "abierto" en el contexto de los LLM

En el módulo anterior construiste sistemas de agentes que consumen APIs de modelos propietarios para razonar, usar herramientas y coordinar sub-agentes. Esas arquitecturas funcionan bien, pero introducen una dependencia estructural: cada llamada al modelo sale del perímetro de tu organización, los datos del usuario atraviesan la red de un tercero, y el costo se acumula por token en cada iteración del loop agéntico. Para equipos que operan agentes de alto volumen, procesan datos que no pueden salir del perímetro corporativo, o necesitan latencia predecible sin variabilidad de red, los modelos locales se convierten en una alternativa legítima o complementaria a las APIs propietarias. Este módulo te da las herramientas técnicas para seleccionar, comprimir, ejecutar, desplegar, especializar y operar esos modelos con la misma rigurosidad de ingeniería que aplicarías a cualquier otro servicio de producción.

El primer problema que encontrarás al explorar modelos locales es la terminología: la mayoría se denominan "open source" pero lo que publican en realidad son solo los pesos entrenados. Esta distinción, que puede parecer filosófica, tiene consecuencias operativas directas sobre qué puedes reproducir, auditar, modificar y redistribuir sin restricciones legales.

Llama 3 de Meta es el ejemplo más citado: sus pesos están disponibles para descarga bajo una licencia comunitaria propia y cualquier organización puede ejecutarlos localmente, pero Meta no publica el pipeline de preentrenamiento, los datos de instrucción usados en el ajuste fino ni los procedimientos exactos de curación. Tampoco es posible reproducir el modelo desde cero porque los datos no están disponibles. En el extremo opuesto del espectro, OLMo del Allen Institute for AI publica los pesos, el código de entrenamiento completo en PyTorch, los datasets de preentrenamiento indexados en Hugging Face, y los checkpoints intermedios de cada etapa del entrenamiento: esto sí se aproxima a la definición rigurosa de open source, permitiendo reproducir el modelo, auditar sesgos en los datos de entrenamiento y detectar vulnerabilidades en el proceso de curación.

Para el AI Engineer que trabaja en producción, esta distinción importa en tres escenarios concretos. Primero, en auditoría de cumplimiento regulatorio: algunos marcos de AI governance (como el EU AI Act para sistemas de alto riesgo) exigen trazabilidad del proceso de entrenamiento que los modelos open weights no proporcionan. Segundo, en reproducibilidad de evaluaciones de seguridad: sin los datos de entrenamiento, es imposible verificar qué memoriza el modelo ni qué sesgos introduce. Tercero, en decisiones de fine-tuning: un modelo con datos de preentrenamiento documentados permite diseñar estrategias de data augmentation más informadas, sabiendo qué dominios están o no representados en el corpus base.

La distinción también afecta a las licencias de forma no obvia. Mistral AI publica sus modelos densos bajo Apache 2.0, lo que permite uso comercial sin restricciones adicionales. Llama 3 usa una licencia comunitaria propia que prohíbe explícitamente usar los outputs del modelo para entrenar modelos competidores y activa restricciones adicionales para servicios con más de 700 millones de usuarios activos mensuales. Un adaptador LoRA entrenado sobre un modelo con licencia restrictiva hereda esas restricciones aunque el adaptador en sí se distribuya bajo MIT: la "apertura" de la capa superior no anula las restricciones de la capa base.

Diferencias técnicas clave

- **Open weights:** se publican los parámetros del modelo en formato SafeTensors o GGUF, pero el pipeline de entrenamiento permanece cerrado o parcialmente documentado.
- **Open source completo:** incluye código de entrenamiento (PyTorch/JAX), datasets indexados y scripts de evaluación reproducibles; OLMo de AI2 y Pythia de EleutherAI son ejemplos representativos.
- **Licencias restrictivas:** modelos como Llama 2/3 prohíben explícitamente uso en servicios con más de 700 millones de usuarios activos mensuales sin acuerdo especial con Meta.
- **Transparencia de datos:** OLMo y RedPajama documentan sus fuentes de datos; la mayoría de los modelos no detallan la composición exacta del corpus de preentrenamiento.
- **Reproducibilidad:** sin el código de entrenamiento y los datos exactos, un modelo open weights no puede ser reproducido desde cero, lo que limita la auditoría de sesgos y vulnerabilidades.

Nota del Arquitecto: *En la práctica, la distinción open weights vs open source rara vez determina la decisión técnica inicial — lo que importa es qué puedes ejecutar, con qué licencia y en qué hardware. La distinción se vuelve crítica cuando el abogado de la empresa pregunta si puedes redistribuir los pesos modificados en un producto comercial. Aprende la licencia del modelo antes de comprometer arquitectura, no después.*

La comprensión de qué significa "abierto" en el contexto de los LLM es el punto de partida obligatorio para cualquier decisión de selección de modelo. Las secciones siguientes exploran las familias de modelos disponibles en 2025, sus características técnicas comparadas y los criterios prácticos para elegir entre ellos según el hardware disponible, el idioma del producto y los requisitos de especialización en dominio.

Módulo 8 – Capítulo 01 – Sección 02

Familias de modelos: Llama, Mistral, Gemma, Phi, Qwen y sus características

Con más de cien mil modelos publicados en Hugging Face a mediados de 2025, el ecosistema de modelos abiertos puede parecer inabarcable. En la práctica, la inmensa mayoría de los modelos utilizables en producción son derivados o variantes de media docena de familias base mantenidas por organizaciones con recursos de preentrenamiento a escala. Conocer estas familias —sus innovaciones arquitectónicas, sus tokenizadores, sus rangos de tamaño y sus comportamientos típicos— es el paso previo a cualquier evaluación comparativa: no todos los modelos de 7B son equivalentes, y elegir la familia incorrecta puede significar meses de trabajo de fine-tuning para recuperar capacidades que otro modelo base ya tenía de forma nativa.

La familia **Llama 3** de Meta marcó un salto cualitativo respecto a las versiones anteriores. Llama 3.1 introdujo Grouped Query Attention (GQA) con ocho cabezas de clave y valor por grupo (en lugar de una por cabeza de consulta como en la Multi-Head Attention original), reduciendo el uso de memoria del KV cache sin penalización significativa de calidad. El vocabulario expandido a 128.256 tokens con byte-pair encoding (BPE) mejora sustancialmente el manejo de idiomas con escrituras no latinas respecto a Llama 2, que tenía apenas 32.000 tokens. Los modelos de la línea 3.1 (8B, 70B, 405B) y 3.2 (con variantes multimodales Vision de 11B y 90B) soportan contextos de hasta 128K tokens, habilitando casos de uso de análisis de documentos largos que los modelos anteriores no podían abordar. La frecuencia de rotación posicional con $\theta=500.000$ permite una extrapolación de contexto más estable que los valores de θ menores usados en modelos anteriores.

Mistral AI tomó una dirección diferente: eficiencia extrema en modelos de tamaño medio. El modelo denso Mistral 7B v0.1 sorprendió al mercado en 2023 al superar a Llama 2 13B en múltiples benchmarks con la mitad de los parámetros, usando una arquitectura con Sliding Window Attention (SWA) de 4.096 tokens por capa y GQA. La familia Mixtral extiende este enfoque con Sparse Mixture of Experts (SMoE): 8 expertos FFN por capa de los cuales solo 2 se activan por token, produciendo modelos como Mixtral 8x7B con 46.7B parámetros totales

pero costo computacional equivalente a un modelo denso de 12.9B. Esta arquitectura impacta directamente en la planificación de hardware: un modelo MoE requiere memoria para alojar todos los pesos de todos los expertos en VRAM (o RAM), pero su throughput de inferencia corresponde al de un modelo mucho más pequeño.

Google Gemma 2 aporta dos innovaciones arquitectónicas relevantes: atención alternada local/global (las capas pares usan atención local de 4.096 tokens, las impares usan atención global) y destilación de conocimiento desde modelos Gemini más grandes durante el entrenamiento. Los modelos de 2B y 9B tienen licencia permisiva para uso comercial, lo que los hace especialmente atractivos para organizaciones que necesitan un modelo pequeño y eficiente sin restricciones de uso.

Phi-3 y Phi-4 de Microsoft demuestran que la calidad del dataset de entrenamiento supera al tamaño en parámetros para muchas tareas. Phi-3-mini de 3.8B, entrenado principalmente en libros de texto sintéticos generados con GPT-4 y código de alta calidad, supera a Llama 2 7B en MMLU y HumanEval siendo significativamente más pequeño. Esta familia es la opción preferida para despliegue en dispositivos con memoria RAM limitada (8-16 GB) donde se necesita razonamiento básico y seguimiento de instrucciones sin la carga de un modelo de 7B.

Qwen 2.5 de Alibaba Cloud destaca por su soporte multilingüe nativo con énfasis en chino, árabe, español y otros idiomas no europeos, y por su ecosistema de variantes especializadas: Qwen2.5-Coder (en tamaños de 0.5B a 32B) para tareas de programación, Qwen2.5-Math para matemáticas con capacidad de cadena de razonamiento, y Qwen2.5-VL para comprensión visual. Para mercados latinoamericanos o con audiencias multilingües, Qwen 2.5 frecuentemente supera a Llama 3 del mismo tamaño en español y portugués.

Características técnicas por familia

- **Llama 3.1/3.2 (Meta):** arquitectura RoPE con $\theta=500.000$, GQA con 8 cabezas KV, contexto de 128K tokens, disponible en 8B, 70B y 405B con soporte multimodal en variantes Vision de 11B y 90B.
- **Mistral/Mixtral (Mistral AI):** ventana deslizante de atención de 4.096 tokens en modelos densos, SMOE con 8 expertos y activación de 2 en Mixtral, tokenizador SentencePiece de 32K tokens.
- **Gemma 2 (Google):** atención alternada local/global, destilación desde Gemini, disponible en 2B y 9B con licencia Apache 2.0 permisiva para uso comercial.

- **Phi-3/Phi-4 (Microsoft):** entrenamiento con datos sintéticos de alta calidad generados con GPT-4, phi-3-mini de 3.8B supera a Llama 2-7B en MMLU y HumanEval.
- **Qwen 2.5 (Alibaba):** soporte multilingüe nativo, variantes especializadas en código y matemáticas, disponible desde 0.5B hasta 72B.

Elegir una familia de modelos no es solo una decisión de benchmarks: implica comprometerse con un tokenizador (que afecta el conteo de tokens para datos en idiomas específicos), una arquitectura de atención (que determina el costo de inferencia en contextos largos) y un ecosistema de modelos derivados de fine-tuning disponibles en Hugging Face. La siguiente sección examina las licencias que aplican a cada familia, que junto con la arquitectura son la segunda dimensión más relevante en la selección inicial de un modelo base.

Módulo 8 – Capítulo 01 – Sección 03

Licencias: MIT, Apache 2.0, Llama Community License y sus restricciones comerciales

Un modelo puede tener excelente rendimiento en todos los benchmarks relevantes y aun así ser inutilizable en tu producto si la licencia prohíbe el caso de uso. La evaluación de licencias de modelos open weights no es un trámite legal de segundo orden: es una decisión de arquitectura que debe tomarse antes de cualquier inversión en fine-tuning, infraestructura o integración. Los equipos que descubren restricciones de licencia después de meses de desarrollo con un modelo base se enfrentan a la opción de reemplazar el modelo (con todos los costos asociados) o negociar una licencia especial con el emisor, proceso que puede tomar meses en organizaciones grandes.

Las licencias de modelos de lenguaje de gran escala no son equivalentes a las licencias de software tradicional: incluso las más permisivas pueden contener cláusulas que van más allá del texto habitual. La **licencia MIT**, usada por modelos como Falcon-7B y algunos checkpoints de Phi-2, permite uso comercial sin restricciones, redistribución y modificación, y no requiere atribución prominente en el producto final. Es el régimen más permisivo y el que genera menos fricciones legales en entornos empresariales. Cuando la librería legal de tu organización pregunta qué licencia tiene el modelo, "MIT" es la respuesta que produce el menor tiempo de revisión.

La **Apache 2.0**, aplicada a Gemma 2 de Google, StableLM y muchos modelos del ecosistema Stability AI, añade dos elementos que MIT no tiene: una cláusula de licencia de patentes (quien distribuye el software concede implícitamente una licencia de patentes sobre las innovaciones contenidas) y la obligación de incluir un archivo NOTICE cuando se redistribuye el código modificado. Apache 2.0 es compatible con uso comercial libre, compatible con GPL v3 pero no con GPL v2, y es el segundo régimen más aceptado en revisiones legales corporativas.

La **Llama Community License**, aplicada a todas las variantes de Llama 2 y Llama 3 de Meta, merece atención especial porque contiene restricciones que no tienen equivalente en

licencias de software tradicional. Primero: prohíbe explícitamente usar los outputs del modelo para entrenar modelos de lenguaje competidores, lo que impacta directamente a organizaciones que pretenden usar Llama 3 como generador de datos sintéticos para entrenar modelos propios. Segundo: cualquier servicio con más de 700 millones de usuarios activos mensuales (MAU) debe obtener una licencia comercial adicional directa con Meta. Para la gran mayoría de las organizaciones este umbral es irrelevante, pero la restricción de uso de outputs para training de competidores afecta a un espectro mucho más amplio de casos. Tercero: cada versión nueva del modelo (Llama 3, 3.1, 3.2) requiere aceptar la licencia actualizada de forma separada; la aceptación de Llama 2 no cubre automáticamente Llama 3.

Un caso especialmente confuso son las **licencias mixtas o en capas**. Algunos modelos como DeepSeek V2 usan MIT para los pesos (permisivo) pero aplican una Acceptable Use Policy (AUP) adicional como términos de servicio que prohíben determinados usos (contenido dañino, aplicaciones de vigilancia masiva). Esta capa adicional no está en el archivo LICENSE del repositorio sino en documentación separada, y legalmente puede prevalecer sobre la licencia MIT en ciertas jurisdicciones. El AI Engineer debe revisar tanto la licencia formal como los términos de uso del repositorio de origen.

El caso de los **pesos derivados** merece mención explícita porque genera errores frecuentes: un adaptador LoRA entrenado sobre un modelo base con licencia restrictiva hereda las restricciones de ese modelo base. Si distribuyes públicamente un adaptador LoRA entrenado sobre Llama 3, ese adaptador está sujeto a la Llama Community License aunque el código de entrenamiento y el adaptador en sí tengan licencia MIT. Esta herencia de licencia es estándar en software pero frecuentemente ignorada en el contexto de modelos.

Aspectos técnicos de las licencias

- **MIT:** sin restricciones de uso, redistribución o modificación; compatible con productos propietarios; no requiere liberar código fuente derivado ni pesos modificados.
- **Apache 2.0:** concede derechos de patente explícitos; requiere NOTICE con cambios realizados; usada por Gemma 2, StableLM; compatible con GPL v3 pero no con GPL v2.
- **Llama Community License:** prohíbe usar outputs para entrenar LLMs competidores; activa restricción de 700M MAU; cada versión requiere aceptación separada.
- **Licencias mixtas:** algunos modelos combinan licencia permisiva para los pesos con AUP adicional; revisar siempre ambas capas de restricciones.
- **Pesos derivados:** el fine-tuning de un modelo con licencia restrictiva hereda las restricciones del modelo base; distribuir un adaptador LoRA sobre Llama requiere

cumplir la Llama Community License.

Nota del Arquitecto: En la práctica, la mayoría de los proyectos empresariales con modelos open weights usan Gemma 2 (Apache 2.0) o Mistral (Apache 2.0) cuando la restricción de redistribución es un requisito, y Llama 3 cuando la licencia comunitaria es aceptable para el caso de uso. No asumir que "open weights" implica "uso sin restricciones" ha evitado más de un problema legal en equipos que descubren tarde las restricciones de la Llama Community License.

La dimensión de licencias completa el marco inicial de selección de modelos: arquitectura, familia y licencia son las tres variables que deben evaluarse antes de la evaluación empírica de calidad. La sección siguiente introduce los leaderboards y herramientas de evaluación que permiten comparar modelos de forma objetiva una vez que el conjunto candidato está definido por criterios de licencia y arquitectura.

Módulo 8 – Capítulo 01 – Sección 04

Leaderboards y evaluación: LMSYS Chatbot Arena, Open LLM Leaderboard y lm-evaluation-harness

La primera decisión operativa al comenzar un proyecto con modelos locales es elegir el modelo base. Esta decisión, que precede a cualquier inversión en fine-tuning o infraestructura, debería estar respaldada por evidencia empírica en la tarea concreta del producto —no solo en benchmarks genéricos. El error más frecuente que cometen los equipos es seleccionar el modelo con el score más alto en el Open LLM Leaderboard sin verificar que ese ranking se transfiere a su tarea, idioma y dominio específicos. Un modelo que lidera MMLU puede quedar tercero en extracción de entidades en español médico; un modelo mediocre en HumanEval puede ser exactamente lo que se necesita para completar consultas SQL en el dialecto propietario de una empresa.

Los **leaderboards públicos** son un punto de partida, no una decisión final. El **Open LLM Leaderboard** de Hugging Face mide rendimiento en benchmarks estandarizados ejecutados de forma reproducible con LM Evaluation Harness: MMLU (conocimiento factual en 57 materias, 14.000 preguntas de opción múltiple), ARC-Challenge (razonamiento científico de nivel escolar avanzado), HellaSwag (coherencia de texto en contextos cotidianos), GSM8K (matemáticas de nivel primaria-secundaria) y, en la versión v2, benchmarks más difíciles como GPQA (preguntas de nivel de doctorado), MuSR (razonamiento multi-paso) y MATH-Hard para reducir la saturación de scores. La fortaleza del Open LLM Leaderboard es la reproducibilidad: cualquier equipo puede replicar los números ejecutando los mismos benchmarks sobre el mismo modelo.

El **LMSYS Chatbot Arena** complementa esta vista con evaluaciones humanas: usuarios reales comparan pares de respuestas de modelos sin saber qué modelo genera cada una, y el sistema calcula un ranking Elo con intervalos de confianza del 95%. Chatbot Arena es más correlacionado con la preferencia humana real que MMLU (las preguntas de opción múltiple no capturan calidad conversacional), pero tiene tres limitaciones sistemáticas: está sesgado hacia inglés conversacional, favorece modelos con buena presentación y formato sobre

modelos técnicamente más precisos, y no permite evaluar tareas de dominio específico (no puedes añadir tus propias preguntas al sistema).

La herramienta que cierra la brecha entre leaderboards genéricos y evaluación en dominio propio es **lm-evaluation-harness** de EleutherAI. Esta librería de Python permite ejecutar más de 200 benchmarks estandarizados localmente sobre cualquier modelo de Hugging Face, GGUF o servido via API compatible con OpenAI, con resultados exactamente reproducibles. La instalación es directa:

```
pip install lm-eval

# Evaluar Llama 3.1 8B en MMLU y GSM8K

lm_eval --model hf \
        --model_args pretrained=meta-llama/Llama-3.1-8B-Instruct \
        --tasks mmlu,gsm8k \
        --batch_size auto \
        --output_path ./resultados/
```

Para modelos servidos localmente con Ollama o vLLM, el backend `local-completions` permite evaluar sin cargar el modelo directamente en la librería:

```
lm_eval --model local-completions \
        --model_args model=llama3:8b,base_url=http://localhost:11434/v1 \
        --tasks mmlu_es,hellaswag_es \
        --batch_size 8
```

Los benchmarks más relevantes según el tipo de tarea son: **MMLU/MMLU-Pro** para conocimiento factual general; **HumanEval** y **MBPP** para generación de código en Python; **GSM8K** y **MATH** para razonamiento matemático; **MT-Bench** para evaluación de instrucción following multi-turno con juez LLM; y **BBH (BIG-Bench Hard)** para razonamiento complejo en tareas que los modelos más pequeños aún no dominan. Para tareas en español, los benchmarks **XNLI** (inferencia de lenguaje natural multilingüe) y **IberBench** cubren razonamiento y conocimiento en el idioma.

La evaluación en dominio propio requiere construir un **golden dataset** antes de seleccionar el modelo base: un conjunto de 100 a 500 preguntas o tareas representativas del producto real con las respuestas esperadas. Este golden set es más valioso que cualquier benchmark

público porque mide exactamente lo que importa. Para construirlo, el proceso recomendado es: (1) extraer ejemplos reales del backlog de casos de uso o consultas de usuarios; (2) incluir explícitamente casos fáciles, casos medios y edge cases conocidos; (3) definir métricas de evaluación para cada categoría (exact match, ROUGE-L, o LLM-as-judge con GPT-4o como árbitro para respuestas abiertas). Con este golden set, la comparación entre modelos candidatos se convierte en una decisión empírica, no en una discusión de opiniones.

Puntos críticos sobre leaderboards y evaluación

- **MMLU:** 14.000 preguntas de opción múltiple en 57 materias; mide conocimiento factual pero no razonamiento complejo; susceptible a contaminación de datos si el modelo fue entrenado con preguntas de MMLU.
- **Open LLM Leaderboard v2:** introdujo GPQA, MuSR y MATH-Hard para reducir la saturación de scores que afectó a la v1 en modelos de 7B o más.
- **LMSYS Chatbot Arena:** calcula Elo con intervalos de confianza del 95%; requiere entre 1.000 y 5.000 comparaciones para estabilizar el ranking; favorece modelos con buena presentación.
- **lm-evaluation-harness:** más de 200 benchmarks ejecutables localmente; soporte para modelos Hugging Face, GGUF via llama.cpp, y APIs OpenAI-compatible; resultados reproducibles.
- **Golden dataset propio:** entre 100 y 500 ejemplos representativos de la tarea real; la métrica más confiable para la selección final del modelo base.
- **Contaminación de benchmarks:** modelos entrenados con preguntas de benchmarks públicos muestran scores inflados; difícil de detectar sin acceso al dataset de preentrenamiento.

Nota del Arquitecto: El golden dataset propio es el único benchmark que no puede ser contaminado, porque nadie más tiene acceso a tus casos de uso reales. Invertir dos o tres días en construirlo antes de seleccionar el modelo base ahorra semanas de fine-tuning para corregir la elección incorrecta. En equipos maduros, este golden set se convierte en el gate de calidad de todos los despliegues futuros.

La evaluación empírica con lm-evaluation-harness y un golden dataset propio es el puente entre los benchmarks genéricos de los leaderboards y la realidad del rendimiento del modelo en producción. La sección siguiente completa el marco de selección con los criterios

cuantitativos de tamaño, idioma, dominio y licencia que, combinados con la evaluación empírica, determinan el modelo base óptimo para cada proyecto.

Módulo 8 – Capítulo 01 – Sección 05

Criterios de selección: tamaño, calidad, licencia, idioma y dominio

Seleccionar un modelo open weights para producción no es una decisión única sino un proceso iterativo que pondera múltiples restricciones simultáneamente: el hardware disponible impone un techo de tamaño de modelo, la licencia puede eliminar candidatos, el idioma y el dominio filtran por calidad, y el resultado final es el modelo que mejor satisface todas las restricciones al menor costo total de inferencia. Entender este proceso como una búsqueda en un espacio de decisión multidimensional —en lugar de una búsqueda del modelo con el benchmark más alto— es el cambio de perspectiva más importante para un AI Engineer que comienza a trabajar con modelos locales.

El **tamaño en parámetros** es el primer filtro porque determina directamente los requisitos de VRAM. Un modelo de 7B en precisión FP16 requiere 14 GB de VRAM; el mismo modelo cuantizado a INT4 con la variante Q4_K_M del formato GGUF ocupa aproximadamente 4 GB, haciendo viable su ejecución en una GPU de consumo de 8 GB. Esta palanca de cuantización amplía enormemente el espacio de modelos viables para cualquier presupuesto de hardware, pero introduce una pérdida de calidad que debe evaluarse empíricamente en la tarea específica (la cuantización se trata en profundidad en el Capítulo 2). La regla operativa más útil es: define el hardware disponible primero, calcula el tamaño máximo de modelo viable con Q4 o Q5, y evalúa la calidad de los mejores modelos en ese rango de tamaño.

El **rendimiento en idioma** es frecuentemente el criterio eliminatorio más importante para productos no angloparlantes, y el que los equipos ignoran con mayor frecuencia. Los rankings de MMLU están dominados por evaluaciones en inglés; el rendimiento en español, árabe, turco o chino varía significativamente entre familias del mismo tamaño. Qwen 2.5 de Alibaba y Mistral-Nemo de 12B muestran rendimiento superior en español y portugués respecto a Llama 3 del mismo tamaño en benchmarks como XNLI y en evaluaciones domain-specific de comprensión lectora. La evaluación en el idioma real del producto, con el golden dataset descrito en la sección anterior, debe preceder siempre la selección final: no asumir que el ranking en inglés se transfiere automáticamente al idioma de producción.

La **especialización en dominio** importa cuando la tarea requiere conocimiento técnico que es minoritario en los datos de preentrenamiento genérico. Para código Python, SQL, Bash o lenguajes de programación comunes, DeepSeek-Coder y Qwen2.5-Coder superan consistentemente a modelos generales de mayor tamaño. Para matemáticas, Qwen2.5-Math y DeepSeek-Math tienen capacidades de razonamiento matemático que los modelos generales de la misma familia no alcanzan. Para conocimiento médico en inglés, MedLM y BioMedLM son bases más sólidas que modelos generales si no se dispone de datos de fine-tuning propios. La pregunta relevante no es "¿qué modelo es mejor en general?" sino "¿qué modelo es mejor para MI tarea específica en MI idioma con MI volumen de datos de fine-tuning disponibles?"

La selección óptima puede visualizarse como la **frontera de Pareto** en el espacio de calidad en la tarea, VRAM requerida y throughput de inferencia. En esa frontera, mejorar una dimensión (por ejemplo, aumentar la calidad pasando a un modelo de 13B) implica empeorar otra (más VRAM requerida, menor throughput). El modelo correcto es el punto de esa frontera que se alinea con las prioridades del producto: si la latencia es crítica, se elige el punto con mayor throughput aunque sacrifique calidad; si la calidad es el diferenciador competitivo, se acepta mayor latencia o mayor costo de hardware; si el costo es la restricción dominante, se maximiza la calidad dentro del presupuesto de VRAM disponible.

Criterios de selección técnicos

- **Tamaño vs hardware:** modelos de 3B-7B son viables en GPUs de consumo de 8-16 GB VRAM; modelos de 13B-70B requieren múltiples GPUs o hardware especializado para inferencia en tiempo real.
- **Rendimiento en idioma nativo:** evaluar con benchmarks específicos del idioma (XNLI, IberBench) o con el golden dataset propio; no asumir que el ranking en inglés se transfiere.
- **Licencia comercial:** verificar explícitamente si la licencia permite uso en el producto, distribución de outputs a usuarios finales y entrenamiento de modelos derivados.
- **Especialización en dominio:** modelos base generales son mejor punto de partida para fine-tuning; modelos instruction-tuned especializados son mejores para zero-shot cuando no hay datos de fine-tuning.
- **Frecuencia de actualización:** familias con releases frecuentes (Qwen, Mistral) permiten incorporar mejoras sin cambiar la arquitectura; familias con ciclos largos ofrecen más estabilidad.

- **Frontera de Pareto:** identificar el punto óptimo en el espacio calidad/VRAM/throughput según las prioridades del producto antes de comprometerse con un modelo base.

Nota del Arquitecto: El criterio de selección más frecuentemente ignorado es la calidad en el idioma de producción. He visto equipos invertir semanas en fine-tuning de Llama 3 8B para un producto en español, solo para descubrir que Qwen 2.5 7B tenía un rendimiento base considerablemente mejor en ese idioma sin ningún fine-tuning. Evalúa en el idioma real del producto antes de comprometerte con un modelo base. Tres días de evaluación empírica ahorran meses de trabajo.

El proceso de selección de modelo base descrito en este capítulo —identificar candidatos por arquitectura y licencia, evaluar con lm-evaluation-harness en benchmarks relevantes y con un golden dataset propio, y seleccionar el punto de la frontera de Pareto que mejor satisface las prioridades del producto— es la base sobre la que todo el trabajo técnico posterior (cuantización, fine-tuning, despliegue) se construye. Un modelo base correctamente seleccionado reduce el esfuerzo de todas las etapas siguientes; uno incorrectamente seleccionado multiplica ese esfuerzo sin garantía de éxito.

Módulo 8 – Capítulo 01 – Sección 06

Cierre: los modelos abiertos han democratizado el acceso a LLM de clase mundial

El ecosistema de modelos abiertos ha recorrido en tres años el camino que la computación en la nube tardó una década en establecer: desde alternativas curiosas con capacidades limitadas hasta opciones legítimas de producción que compiten directamente con modelos propietarios en la mayoría de las tareas de negocio. Llama 3.1 405B supera en múltiples benchmarks a GPT-3.5-turbo, el modelo que definió el estado del arte en 2023, y lo hace con pesos que cualquier organización puede descargar y ejecutar en su propia infraestructura. La disponibilidad de modelos especializados en código, matemáticas, idiomas específicos y razonamiento estructurado permite que los equipos de ingeniería elijan el modelo más eficiente para cada subtarea en lugar de depender de un único proveedor de API general.

Este cambio transforma el rol del AI Engineer de forma estructural. Ya no se trata únicamente de consumir APIs con parámetros bien documentados y SLAs garantizados por terceros: se trata de seleccionar modelos como artefactos de primera clase en la arquitectura del sistema, evaluarlos empíricamente contra criterios de producción propios, comprimirlos para el hardware disponible, especializarlos con datos del dominio y operarlos con todas las implicaciones de governance y ciclo de vida que eso conlleva. La selección de modelo —que este capítulo ha introducido como un proceso multidimensional de evaluación empírica— es solo la primera de muchas decisiones técnicas que se abordan en los capítulos siguientes.

El arco del módulo sigue la secuencia lógica del trabajo de un AI Engineer que incorpora modelos locales a su stack: seleccionar el modelo correcto (este capítulo) → comprimir su footprint de memoria mediante cuantización (Capítulo 2) → ejecutarlo localmente con llama.cpp y Ollama (Capítulo 3) → planificar el hardware apropiado (Capítulo 4) → desplegarlo en producción con motores de alto throughput (Capítulo 5) → especializarlo con fine-tuning eficiente cuando el prompting no es suficiente (Capítulo 6) → alojarlo en infraestructura cloud GPU escalable (Capítulo 7) → optimizar su rendimiento de inferencia (Capítulo 8) → gestionar su ciclo de vida con governance de producción (Capítulo 9) →

integrarlo en arquitecturas híbridas con modelos en la nube (Capítulo 10). Cada capítulo asume el conocimiento del anterior y añade una capa de complejidad operativa.

Idea central

El verdadero poder de los modelos abiertos no está solo en el ahorro de costos de API, sino en la capacidad de personalizar, auditar, desplegar con privacidad y evolucionar el modelo de acuerdo con las necesidades específicas del producto. La selección informada del modelo base es la decisión que hace posible todo lo demás.

"The most powerful technologies in history have been those that gave everyone access to what was once reserved for the few." — Tim O'Reilly, fundador de O'Reilly Media, sobre el impacto democratizador del software abierto en infraestructura tecnológica.